

Sistemas de Informação
Conecta Banco de Dados com POO - 2026.1

PROJETO

Conecta Voluntários

Equipe

Arthur Valença Florindo – 2513080062
Gabriel Nathan Ferreira – 2513080092
Júlio César Gonçalves dos Santos – 2513080018
Larissa Farias Cavalcante - 2513080004

1. Introdução e Objetivos

1.1 Introdução

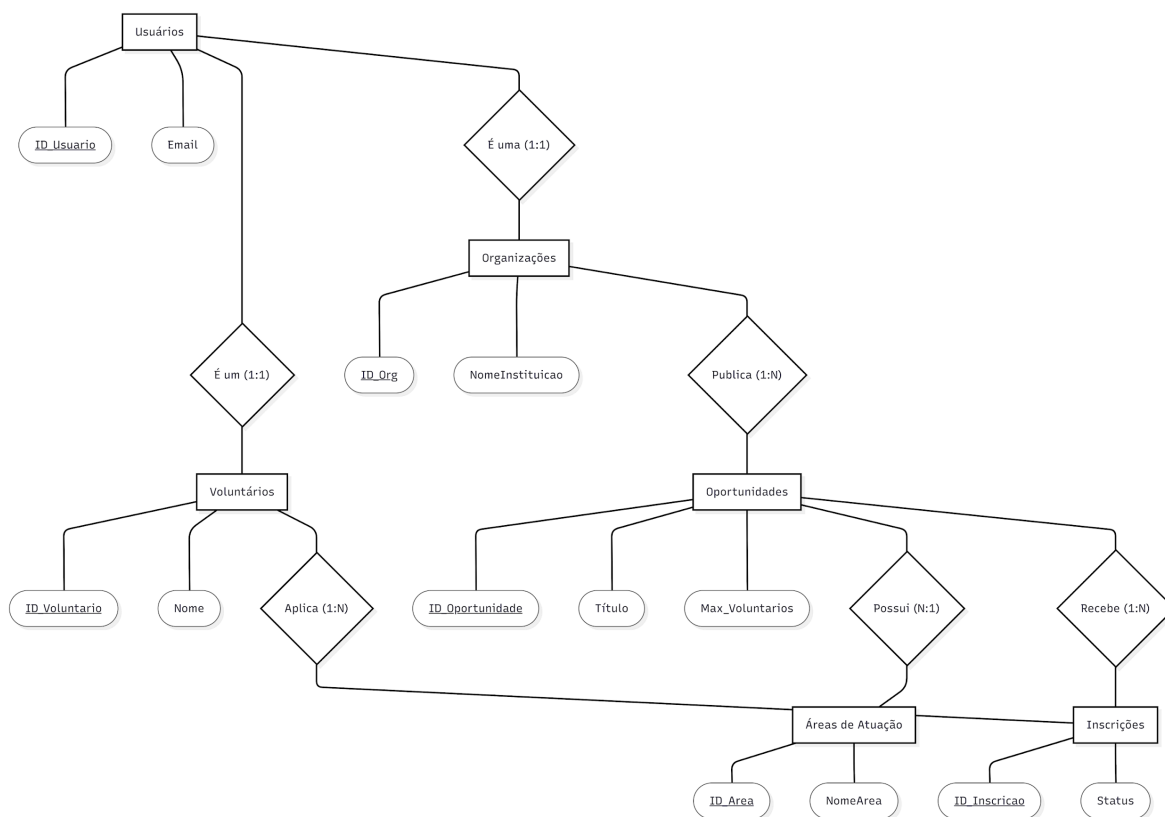
O gerenciamento de ações sociais e voluntariado ainda enfrenta muitas barreiras operacionais. Muitas instituições e ONGs dependem de processos manuais ou ferramentas informais para divulgar suas vagas e selecionar ajudantes. Na outra ponta, cidadãos dispostos a colaborar encontram dificuldades para localizar oportunidades que se alinhem ao seu perfil e disponibilidade.

O sistema **Conecta Voluntários** foi idealizado para sanar esse problema. A proposta central é estruturar uma plataforma que integre e automatize o fluxo de informações entre voluntários e organizações, cobrindo desde o mapeamento de áreas de interesse até o controle rigoroso de inscrições e a consolidação das horas trabalhadas no histórico de impacto.

1.2 Objetivos

- **Objetivo Geral:** Desenvolver e implementar a modelagem de um banco de dados relacional capaz de suportar as operações do sistema Conecta Voluntários, garantindo a consistência das regras de negócio diretamente na camada de dados.
- **Objetivos Específicos:**
 - Elaborar a modelagem conceitual (Diagrama de Peter Chen) e o modelo lógico para garantir o correto mapeamento do escopo do sistema.
 - Implementar rotinas armazenadas (*procedures e functions*) para automatizar processos críticos, como o fluxo seguro de inscrições.
 - Desenvolver restrições através de gatilhos (*triggers*) para impedir inconsistências nas vagas disponíveis e automatizar a atualização do histórico de impacto.
 - Criar uma estrutura de auditoria interna por meio de logs automáticos para monitorar as ações dos usuários no sistema.

2. Diagrama de Entidade Relacional (DER)



3. Descrição das Tabelas (Dicionário de Dados)

3.1 Tabela: Usuarios

- **Descrição:** Armazena as credenciais de acesso de todas as pessoas ou entidades cadastradas na plataforma. É a tabela base para o controle de segurança e autenticação.
- **Campos:**
 - ID_Usuario (Chave Primária): Identificador único do usuário.
 - Email: Endereço eletrônico utilizado para login (com restrição de unicidade).
 - SenhaHash: Senha criptografada para garantir a segurança.
 - TipoUsuario: Define o nível de acesso no sistema (ex: 'Voluntario', 'Organizacao', 'Admin').

3.2 Tabela: Voluntarios

- **Descrição:** Contém as informações de perfil dos cidadãos que desejam prestar serviços voluntários.
- **Campos:**
 - ID_Voluntario (Chave Primária e Estrangeira): Vinculado diretamente à tabela Usuarios.
 - Nome: Nome completo do voluntário.
 - CPF: Cadastro de Pessoa Física (para validação interna).
 - Cidade: Localização do voluntário para sugerir vagas próximas.

3.3 Tabela: Organizacoes

- **Descrição:** Cadastro das instituições, ONGs ou entidades filantrópicas que publicam as oportunidades de ajuda.
- **Campos Principais:**
 - ID_Org (Chave Primária e Estrangeira): Vinculado diretamente à tabela Usuarios.
 - NomeInstituicao: Razão social ou nome fantasia da entidade.
 - CNPJ: Cadastro Nacional da Pessoa Jurídica para fins de validação institucional.

3.4 Tabela: Areas_Atualizacao

- **Descrição:** Tabela de domínio que lista as possíveis frentes de ajuda disponíveis no sistema (ex: Educação, Meio Ambiente, Saúde, Proteção Animal).
- **Campos Principais:**
 - ID_Area (Chave Primária): Identificador da área de atuação.
 - NomeArea: Nome descritivo da categoria de serviço.

3.5 Tabela: Oportunidades

- **Descrição:** Registra as vagas de voluntariado abertas pelas organizações. Centraliza as regras de ocupação.
- **Campos Principais:**
 - ID_Oportunidade (Chave Primária): Identificador único da vaga.
 - ID_Org (Chave Estrangeira): Aponta para a organização que criou a oportunidade.
 - ID_Area (Chave Estrangeira): Classifica a oportunidade em uma área de atuação específica.
 - Titulo e Descricao: Informações detalhadas sobre a atividade.
 - Max_Voluntarios: Limite máximo de ajudantes permitidos.
 - Vagas_Ocupadas: Contador dinâmico gerenciado pelo banco para controlar a lotação.

3.6 Tabela: Inscricoes

- **Descrição:** Tabela associativa que gerencia as solicitações dos voluntários para participarem de uma oportunidade específica.
- **Campos Principais:**
 - ID_Inscricao (Chave Primária): Identificador da inscrição.
 - ID_Voluntario (Chave Estrangeira): Identifica o voluntário solicitante.
 - ID_Oportunidade (Chave Estrangeira): Identifica a vaga desejada.
 - Status: Estado atual da inscrição (ex: 'Pendente', 'Aprovado', 'Concluido', 'Cancelado').
 - DataInscricao: Registro da data e hora em que a inscrição foi feita.

3.7 Tabela: Historico_Impacto

- **Descrição:** Tabela gerada automaticamente para consolidar o total de horas dedicadas e as metas cumpridas pelo voluntário após a conclusão de uma atividade.
- **Campos Principais:**
 - ID_Historico (Chave Primária): Identificador do registro de histórico.
 - ID_Voluntario (Chave Estrangeira): Vincula as horas ao voluntário correspondente.
 - Horas_Trabalhadas: Quantidade de horas computadas para aquela ação social.

3.8 Tabela: Logs_Sistema

- **Descrição:** Destinada à trilha de auditoria e segurança. Grava de forma automatizada (via gatilho) qualquer alteração ou evento crítico realizado na tabela de usuários.
- **Campos Principais:**
 - ID_Log (Chave Primária): Identificador do evento de log.
 - Acao: O tipo de operação executada (ex: 'INSERT', 'UPDATE', 'DELETE').
 - TabelaAfetada: Nome da tabela onde o evento ocorreu.
 - DataHora: Timestamp exato da execução da query para fins de rastreabilidade.

4. Explicação detalhada das Procedures, Funções e Triggers

4.1 Stored Procedures (Procedimentos Armazenados)

- **sp_RegistrarInscricao** Esta procedure é responsável por centralizar o fluxo de inscrição de um voluntário em uma oportunidade. Ao receber o ID_Voluntario e o ID_Oportunidade, o procedimento inicia uma transação segura. Primeiramente, realiza uma consulta de validação para verificar se a oportunidade ainda possui vagas disponíveis ($Vagas_Ocupadas < Max_Voluntarios$). Se houver vaga, ela insere o registro na tabela Inscricoes com o status inicial 'Pendente' e incrementa o contador de vagas na tabela de oportunidades. Caso contrário, ela dispara um erro customizado abortando a operação.

4.2 Functions (Funções)

- **fn_CalcularTotalHorasVoluntario** Uma função escalar que recebe como parâmetro de entrada o identificador exclusivo do voluntário. Ela executa uma varredura agregada (SUM) na tabela Historico_Impacto, filtrando pelas atividades cuja conclusão foi devidamente homologada. A função retorna um valor inteiro representando a carga horária consolidada do cidadão, métrica crucial para a emissão de certificados e relatórios de impacto social.

4.3 Triggers (Gatilhos Automáticos)

- **tg_AuditoriaUsuarios** Configurado para agir de forma transparente após qualquer evento de inserção, atualização ou exclusão (AFTER INSERT/UPDATE/DELETE) na tabela de credenciais (Usuarios). O gatilho captura o usuário do banco que realizou a operação, o tipo de comando SQL executado e o timestamp exato, gravando os metadados diretamente na tabela Logs_Sistema. Isso assegura conformidade com boas práticas de segurança e LGPD.
- **tg_AtualizarHistoricoImpacto** Disparado imediatamente AFTER UPDATE na tabela de Inscricoes. Quando o status de uma inscrição é modificado para 'Concluído', este gatilho intercepta a transação e injeta automaticamente um novo registro na tabela Historico_Impacto, calculando as horas associadas àquela oportunidade específica sem a necessidade de intervenção humana ou código externo.

5. Prints dos Testes e Logs

Execução da Procedure de Registro de Inscrição Pendente

332 • `CALL sp_RegistrarInscricao(2, 2);`

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Titulo	Vagas_Ocupadas		
▶	Apoio em Hortas	0		

Aprovação de Inscrição por ID via Stored Procedure

334 • `CALL sp_AprovarVoluntario(6);`
335

Output				
Action Output				
#	Time	Action	Message	
✓ 1	21:17:56	CALL sp_AprovarVoluntario(6)	0 row(s) affected	

Finalização de Participação e Computação de Carga Horária

335
336 • `CALL sp_FinalizarParticipacao(6, 12);`
337

Output				
Action Output				
#	Time	Action	Message	
✓ 1	21:18:55	CALL sp_FinalizarParticipacao(6, 12)	0 row(s) affected	

Consulta Estruturada à Tabela de Controle de Inscrições

333

334 • `SELECT ID_Inscricao, ID_Voluntario, Status FROM Inscricoes;`

335

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	ID_Inscricao	ID_Voluntario	Status
▶	1	1	Concluido
	2	2	Pendente
*	NULL	NULL	NULL

Verificação Física de Dados no Histórico de Impacto

350

351 • `SELECT Titulo, Vagas_Ocupadas FROM Oportunidades WHERE ID_Oportunidade = 2;SELECT * FROM HistoricoImpacto WHERE ID_Voluntario = 2;`

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	ID_Historico	ID_Voluntario	ID_Oportunidade	Horas_Trabalhadas	DataConclusao
▶	3	2	2	12	2026-05-21
*	NULL	NULL	NULL	NULL	NULL

6. Conclusão e Resultados

6.1 Desafios do Projeto

O principal desafio durante o desenvolvimento do projeto foi garantir a integridade dos dados diante de regras de negócio concorrentes. A modelagem exigiu atenção especial na normalização das tabelas até a 3ª Forma Normal (3FN), principalmente na resolução do relacionamento muitos-para-muitos (\$N:M\$) entre Voluntários e Áreas de Atuação, o que demandou uma tabela associativa bem estruturada para evitar redundâncias.

Outro ponto complexo foi o desenvolvimento da lógica de controle de vagas dentro da *stored procedure* de inscrição. Foi necessário estruturar validações rígidas para assegurar que o limite máximo de voluntários de uma oportunidade não fosse violado em acessos simultâneos, o que exigiu um entendimento aprofundado sobre transações e tratamento de erros no MySQL.

6.2 Resultados Obtidos

Os resultados obtidos foram bastante satisfatórios, consolidando um banco de dados funcional e totalmente integrado ao escopo proposto. Conseguimos traduzir com sucesso os requisitos do sistema para o modelo de Peter Chen e, posteriormente, para o modelo físico.

A implementação de *triggers* e rotinas armazenadas provou ser eficiente, permitindo que o banco de dados processe regras complexas de forma independente da aplicação — como a inserção automática no histórico de impacto logo após a conclusão de uma inscrição. Por fim, a inclusão do subsistema de logs garantiu uma camada essencial de segurança e auditoria, resultando em um projeto sólido, escalável e aderente às práticas de engenharia de dados.